JAVASCRIPT

JAVASCRIPT DE MENOS A MAS

EJERCICIOS Y APUNTES PRÁCTICOS DE JAVASCRIPT

AUTOR @TITOR-OOPART
YOUTUBE: @ReadToRun
GITHUB BOOK: @ReadToRun

4 de mayo de 2024

Índice general

1. Que es JavaScript **5**

2. Que con el nombre de JavaScript **7**
2.1. Muchas caras 7
2.2. React.createElement() 9
2.3. JSX 10
2.4. On Transpilation 11
2.5. Custom Components 12

3. Hola mundo **13**
3.1. Configuracion 13

4. La vida de un Componente **15**
4.1. Una funcion Custom Component 15
4.2. Custom Component version JSX 16
4.3. Un componente custom en una clase 16
4.4. Que sintaxis Usar? 17
4.5. Propiedades 18
4.6. Propiedades en Function Components 18
4.7. Default Properties 19
4.8. State 21
4.9. Textarea Component 21
4.10. Hacerlo Stateful 22
4.11. Una nota sobre DOM Events 23
4.12. Event Manejado en los viejos tiempos 23
4.13. Manejo de Eventos en React 24
4.14. Sintaxis de manejo de Event 25
4.15. Props Versus State 27
4.16. Props en estado inicial: Un Antipatron 27
4.17. Accediendo al Componente desde afuera 28
4.18. Ciclo de vida Methodos 28
4.19. lifecycle ejemplo: Log it All 29
4.20. Paranoid State Protection 30
4.21. lifecycle ejemplo: Usando un componente hijo 32
4.22. Rendimiento Win: Prevenit Actualizacion de componentes 33

5. Excell Una fantastica Tabla de Componentes **37**
5.1. Primero los Datos 37
5.2. Table Headers Loop 38
5.3. Agregando Content $< td >$ 39
5.4. Prop Types 41
5.5. Sorting 43
5.6. Sorting UI Cues 45
5.7. Editable cell 48
5.8. Conclusiones y Virtual DOM Diffs 51
5.9. Search 52
5.10. State y UI 52
5.11. Filtrando el contenido 52
5.12. Actualizar el metodo save() 53
5.13. Como podrias mejorar el Search? 54
5.14. Instant Replay 54
5.15. Limpiando manejador de Eventos 55
5.16. Cleaning Solution 56
5.17. Puedes mejorar el Replay? 56
5.18. Una Implementacion Alternativa 56
5.19. Download los datos de la tabla 57

6. Functional Excell **59**
6.1. Un pequenio recordatorio: Function versus Class Components 59
6.2. Renderisando el Data 59
6.3. The State Hook 60

Capítulo 1

Que es JavaScript

Es importante entender como JS esta organizado y mantenido, es una buena manera de aprender JS
JavaScript esta organizado en tres pilares principales: scope/closures, prototypes/objects y types/coercion JS es un lenguaje sofisticadcon muchos features y capabilities. Pero todo JS es encontrado en estos tres pilares fundamentales.

Capítulo 2

Que con el nombre de JavaScript

El nombre JavaScript es probablemente es el error y la falta de entendimiento del lenguaje de programacion
Esta relacionado con Java? es solo es script de Java? es solo para escribir scripts o escribir programas?
Este lenguaje fue originalmente diseniado para atraer a la audiencia programadores Java, y porque la palabra script fue popular el tiempo para referirse a programas ligeros. Este scripts de poco peso sera el primero para ponerlos dentro de paginas en la web
La manera como JS es implementado para los navegadores web es en toda practica lo unico que realmente importa.
Varias ambientes JS como motor browser JS, Node.js, etc agregan APIs dentro el alcance global de su JS programa que da capacidades especificas-ambiente, como hacer disponible salte una alerta-syle box en el browser del usuario.
Es un hecho una amplia gama de JS APIs, como fetch(..), getCurrentLocation(..) y getUserMedia(..) son todas las APIs web que se ven como JS, En Node.js podemos acceder a cientos de metodos API desde varios modulos built-in como fs.write(..)
Otro ejemplo comun es console.log(..) y todos los otros metodos cosole.* Esto no esta especificado en JS porque su utilidad esta definida por muchos
Entonces alert(..) y console.log(..) no estan definidos por JS pero parecen como JS. Son funciones y objects methods y obedecen la sintaxis de JS. El comportameinto detras de ellos esta controlado por el ambiente corriendo el motor JS
La consola de desarrollador no esta tratando pretender ser un compilador motor JS manejado un archivo .js esta tratando de hacerlo facil no es rasonable esperar una herramienta para manejar las dos coas de manera igual
No creas que el ambiente que ves en el developer console representa exactamente a la semantica JS; para esto, lee las especificaciones. En lugar pensar que la consola es un ambiente JS-amigable

2.1. Muchas caras

El termino paradigma en programacion hace referencia amplia (la mayoria universal) set de pensamiento y asercarse a la estructuracion del codigo Dentro de un paradigma, existen innumerables variaciones. de estilo y forma que distinguen a los programas, incluidos los de bibliotecas y marcos menos diferentes que dejan su singularidad firma en cualquier código dado.
Pero no importa como puede ser un programa individual, la gran imagen divide entre

paradigmas son mas evidentes lo primero de cualquier programa
Tipicamente categorias de code paradigm-level incluye procedural, object-oriented(OO/classes), y functional(FP):

- Procedural style organiza el codigo en un top-down, regresion linear atravez de pre determinado set de operaciones usualmente collectado junto en unidades relacionadas llamadas procedurales
- OO syle, organiza el codigo collectando logica y data juntas dentro de nudades llamadas classes
- FP sytle, organiza codigo dentro funciones (puros calculos como opuesto a procedimientos) y la adaptacion de estas funciones como valores

Paradigmas no son correctas ni incorrectas. Estan orientados a guiar y moldear como programadores manejaran los problemas y soluciones JavaScript es un lenguaje multi-paradigma. Puedes escribir procedural, class-oriented o FP syle code, y puedes hacerlo linea por linea basado en tu decicion, en lugar de ser forsado dentro una todo o nada

```
<!doctype html>
<html>
  <head>
    <title>React hola mundod</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app"></div>
    <script src="../reactbook/react/react.js"></script>
    <script src="../reactbook/react/react-dom.js"></script>
    <script>
      ReactDOM.render(
        React.createElement("h1", null, "Hola Mundo!"),
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

Podemos cambiar h1 por otro tipo de etiqueta y asi ir conociendolas Notemos que el etiqueta con el id app sera actualizada con el componente h1 que se crea con el ReactDOM Hay un concepto de componentes. Contruyes tu UI usando componentes y los combinas en la manera que veas. En tus aplicaciones puedes crear tus custom components, React provee envoltorios al rededor de HTML DOM elemtns. Tu puedes usar estos con la funcion Recat.CreateElement En este pribmer ejemplo puedes ver el uso de h1 que corresponde a <h1>en HTML
Finalmente veras el buen document.getElementById('app') DOM access Usaras esto para decirle a React donde sera localisada la aplicacion en la pagina. Esto es el puente crusando de la manipulacion del DOM a React.
Una ves crusas el puente del DOM a React, no te preocuparas sobre la manipulacion del DOM por que React hace la tralacion del componente al plataforma(DOM, canvas, native app) Es un hecho no que preocuparte sobre el DOM es una de las cosas geniales de React. Te preocuparas mas sobre la composicion de componentes y sus datos deja a React tomar y actualizar el DOM de manera mas eficiente.

2.2. React.createElement()

Como concoes puedes usar un numero de elemtnos HTML como React componentes via React.createElement() veamos mas de cerca esta API
Analizando el parametro para createElement el primero es el tipo de elemento a ser creado. El segundo donde escribimos null es un objeto que especifica una propiedad (DOM attribute) que tienes que pasarle a tu componente por ejemplo

```
  React.createElement(
  'h1',
  {
    id: 'my-heading'
  },
  'Hello world!'
  )
```

El HTML generado por este ejemplo en la etiqueta h1 tendra un tributo id='my-heading'
El tercer parametro 'Hello world' define el hijo del componente. Este ejemplo simple es un hijo texto donde pueden ir otros componentes como en este ejemplo y puedes tener tantos hijos como quieras

```
  React.createElement(
  'h1',
  {
    id: 'my-heading'
  },
  React.createElement('span, null, 'Hello'),
  ' world!',
  'This is good day!'
  )
```

```
   <!doctype html>
<html>
  <head>
    <title>React hola mundod</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app"></div>
    <script src="../reactbook/react/react.js"></script>
    <script src="../reactbook/react/react-dom.js"></script>
    <script>
      ReactDOM.render(
        React.createElement(
          "h3",
          null,
          React.createElement("span", null, "Hola"),
          " Mundo",
          " Es un gran dia!",
        ),
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

2.3. JSX

Cuando empezamos a anidar componentes, rapidamente terminas con muchas funciones llamadas y parentesis para trackerarlos. Para hacer las cosas mas faciles puedes usar el sintaxis JSX JSX es un poco controversial, las personas a menudo lo encuentras repulsivo diciendo(ugh, XML en mi JavaScript) pero despues es indispensable Para esto necesitamos descargar babel en nuestro folder react

```
\$ curl -L https://unpkg.com/babel-standalone/babel.min.js > ~/
reactbook/react/babel.js
```

Aqui tenemos un ejemplo

```
<!doctype html>
<html>
  <head>
    <title>Hello React</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app">
      <!-- my app renders here -->
    </div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script type="text/babel">
      ReactDOM.render(
        <h1 id="my-heading">
          <span>
            Hello<em> Wonderful </em>
          </span>
          wold!
        </h1>,
        document.getElementById("app"),
      );
    </script>
  </body>
</html>!doctype html>
<html>
  <head>
    <title>Hello React</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app">
      <!-- my app renders here -->
    </div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script type="text/babel">
      ReactDOM.render(
        <h1 id="my-heading">
          <span>
            Hello<em> Wonderful </em>
          </span>
          wold!
        </h1>,
```

```
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

Ahora otro ejemplo
01.04.hellojsx.html

```
<!doctype html>
<html>
  <head>
    <title>Hello React</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app">
      <!-- my app renders here -->
    </div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script type="text/babel">
      ReactDOM.render(
        <h1 id="my-heading">
          <span>
            Hello<em> JSX </em>
          </span>
          wold!
        </h1> ,
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

2.4. On Transpilation

Genial que ya entendiste JSX y Babel para trabajar
JSX es una tecnologia separada de React y es completamente opcional. Como puedes ver El proceso de transpilation es un proceso de tomar el codigo y reescribirlo en para convertirlo en el mismo resultado pero usando sintaxis que es entendido por los antiguos buscadores. Es diferentes que usar polyfill. Un ejemplo de un polyfill es agregar un metodo al Array.prototype como map(), que fue introducido en ECMAScript5 y haciendolo fucnionar en un browser que soporta solo ECMAScript3. Un polyfill es una solucion en puro JavaScript es una buena solucion cuando agregamos nuevos metodos a objetos existentes o implementamos nuevos objetos(como JSON). Pero no es suficiente cuandod cuando nuevas sintaxis es introducida dentro del lenguaje. Cualquier nueva sintaxis en los ojos del browser que no soporte es invalida y lanzar erroers. No hay manera de polyfill. Nueva sintaxis por esa rason requiere una compilcacion paso (transpilation) sera transformado antes de ser servido en el navegador
Transpiling JavaScript esta siendo mas y mas comun como programadores que quieren usar el ultimo JavaScript(ECMAScript) sin esperar por navegadores lo implementen

2.5. Custom Components

En este punto ya sabes como

- Configurar una librery React para experimentacion y aprendisaje solo usar tags <script>
- Renderisar un componente React en un DOM ReactDOM.render(queReact, donde-DOM)
- Usar componentes built-in cual estan envueltos sobre elemntos regulares DOM React.createElement(elemento, atributo, contenido, hijo)

Ahora en el siguient capiutlo aprendemos sobre Componentes custom

Capítulo 3

Hola mundo

3.1. Configuracion

Creamos una carpeta para el la practica de este libro y otra para react dentro del primero

```
$ mkdir reactbook
$ mdir reactbook/react
```

A continuacion necesitamos agregar dos archivos uno para React el otro para el ReactDom

```
$ curl -L https://unpkg.com/react@17/umd/react.development.js > ~/
   reactbook/
react/react.js
$ curl -L https://unpkg.com/react-dom@17/umd/react-dom.development.js >
    ~/react-
book/react/react-dom.js
```

Borra el @17 si ya contabas con una version de nodejs

```
<!doctype html>
<html>
  <head>
    <title>Hello React</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app">
      <! -- my app renders here -->
      <h1>Hello World!</h1>
    </div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script>
      ReactDom.render(
        React.createElement("h1", null, "HelloWorld!"),
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

Aqui podemos ver diferentes cosas

1. El documento empieza siempre con <!Doctype html>

2. El elemento de bloque padre seria <html></html>
3. Los otros dos elementos padres serian <head></head>y <body></body>

a

Capítulo 4

La vida de un Componente

Ahora que sabemos como usar DOM componentes aprendamos a crear nuestros componentes
Hay dos maneras para definir un componente ambos daran el mismo resultado pero usando diferentes sintaxis

- Usando una funcion(Componentes creados de esta manera se refieren como function components)
- Usando una class clase que es extendida de React.Component(comunmente referida como clase componente)

4.1. Una funcion Custom Component

Aqui un ejemplo

```
const MyComponent = function() {
    return 'I am so custom'
  };
```

Espera esta es solo una funcion pero retorna la UI que quieres

```
const MyComponent = function() {
    return React.createElement('span', null, 'I am so custom');
  };
```

Con el componente creado en el capitulo anterior podemos agregar el nuevo de la siguiente manera:

```
<!doctype html>
<html>
  <head>
    <title>Hello React</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app">
      <!-- my app renders here -->
    </div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script>
```

```
      const MyComponent = function () {
        return React.createElement("span", null, "I am so custom");
      };
      ReactDOM.render(MyComponent(), document.getElementById("app"));
    </script>
  </body>
</html>
```

4.2. Custom Component version JSX

Se puede hacer el mismo ejemplo ahora usando babel y JSX

```
<!doctype html>
<html>
  <head>
    <title>Hello React</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app">
      <!-- my app renders here -->
    </div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script type="text/babel">
      const MyComponent = function () {
        return <span>I am so custom</span>;
      };
      ReactDOM.render(<MyComponent />, document.getElementById("app"));
    </script>
  </body>
</html>
```

4.3. Un componente custom en una clase

La segunda manera para crear un componente es definirlo en una clase que es extendida de React.Component e implementa una funcion render()
02.03.custom-class.html

```
<!doctype html>
<html>
  <head>
    <title>Hello React</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app">
      <!-- my app renders here -->
    </div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    // <script type="text/babel">
    // Con JSX
    // <script type="text/babel">
```

```
    <script>
      class MyComponent extends React.Component {
        render() {
          return React.createElement("span", null, "I am so custom");
          // Con JSX
          // return <span>I am so custom</span>;
        }
      }
      ReactDOM.render(
        React.createElement(MyComponent),
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

Aqui el ejemplo con clases en sintaxis JSX
02.04.custom-class-jsx.html

```
<!doctype html>
<html>
  <head>
    <title>Hello React</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app">
      <!-- my app renders here -->
    </div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script type="text/babel">
      class MyComponent extends React.Component {
        render() {
          return <span>I am so custom JSX</span>;
        }
      }
      ReactDOM.render(<MyComponent />, document.getElementById("app"));
    </script>
  </body>
</html>
```

4.4. Que sintaxis Usar?

Seguro esperas saber que opcion usar (JSX vs javascript puro, class component vs function component), JSX es el mas comun. amenos que no te guste sintaxis xml en tu codigo JavaScript, el camino de menos resistencia y de menor escritura serai escojer JSX Este libro usa JSX desde ahora, Por que despues hablaremos sobre la manera no-JSX? Bueno deberias asaber que hay otra manera y tambien que JSX no es magico pero tiene una sintaxis corta que transforma XML en plain llamadas JavaScript function como React.createElement() antes de enviarlo al codigo del navegador
Que sobre class vs function components? Si estas agusto con OOP puedes y te gusta las clases ve por ese camino, Function components son un poco mas ligero en el CPU de la computadora y requiere menos escritura de codigo.

tambien se siguiente mas nativo a JavaScript. Actualmente classes no existen en una version reciente del lenguaje JavaScript;
Historicamente, tan rapido como React es concernado, los componentes de fucnioens no estan disponible para hacer todo lo que las clases hacen. Hasta la invencion de los hooks, que es la manera como tendras las dos maneras. Para el futuro uno puede solo especular, pero es como que React movera mas y mas components functions. Como sea es favorable aun que no nos guste que las class components van a ser deprecadas alguna tiempo pronto.
Este libro te enseniara las dos manears y no decidira por vos, eso dependera de tu preferencia
Aunqeu en la documentacion muestra mas ejemplo como class components te haras familiar con ambas opciones y podras entender todo el codigo presente y no ocnfundirte

4.5. Propiedades

Renderisando hard-coded UI en tu custom componetns es perfecto bien tiene sus usos . Pero los componentes puede tomar propiedades y rendereisar o comportarse differente, dependiendo de los valores de las propiedades. Piensa sobre el elemento <a>en HTML y como este actua diferente basado en el valor del atributo href
La idea de propiedades en React es similar(y esta es la JSX sintaxis)
En una clase component todas las propiedades estan disponible via this.props object, Veamos un ejemplo

```
class MyComponent extends React.Component {
  render() {
    return <span>My name is <em>{this.props.name}</em></span>;
  }
}
```

Pasando un valor de la propiedad del nombre cuando renderisamos el component se parece a esto

```
      ReactDOM.render(
        <MyComponent name="Titor" />,
        document.getElementById("app"),
      );
```

4.6. Propiedades en Function Components

En una funcion component, No hay esto (en JavaScript strict mode) o esto refiere al objeto global(en non-scrit, sloppy mode)en lugar de this.props, tu tendras un props object pasado a tu funcion como primer argumento

```
      const MyComponent = function (props) {
        return (
          <span>
            I am <em>{props.name}</em>
          </span>
        );
      };
```

```
      ReactDOM.render(
        MyComponent({ name: "Titor" }),
        document.getElementById("app"),
      );
```

Un patron comun es usar JavaScript destructuring assignment y asignar el valor de la propiedad a la variable local. En otras palabras como el ejemplo

```
// 02.07.props.destructuring.html
      const MyComponent = function ({ name }) {
        return (
          <span>
            I am <em>{name}</em>
          </span>
        );
      };
      ReactDOM.render(
        <MyComponent nam="Titor" />,
        document.getElementById("app"),
      );
```

Aqui un ejemplo con multiples propiedades

```
// 02.08.props.destruct.multi.html
      const MyComponent = function ({ name, lastname, job }) {
        return (
          <span>
            I am{" "}
            <em>
              {name} {lastname}
            </em>{" "}
            the <em>{job}</em> and I love technology
          </span>
        );
      };
      ReactDOM.render(
        <MyComponent name="Titor" lastname="Oopart" job="Software
    Engineer" />,
        document.getElementById("app"),
      );
```

4.7. Default Properties

Tu componente podria offercer un numero de propiedades, pero algunas veces un poco de las propiedades podrian tener valores por default que funcionan bien para la mayoria de los casos.
Puedes especificar propiedades por default usando ddefaultProps property para ambas funciones y class components

```
const MyComponent = function ({ name, lastname, job }) {
        return (
          <span>
            I am{" "}
            <em>
              {name} {lastname}
            </em>{" "}
```

```
            the <em>{job}</em> I love technology
          </span>
        );
      };
      MyComponent.defaultProps = {
        job: "Engineer",
      };
      ReactDOM.render(
        <MyComponent name="Titor" lastname="Oopart" />,
        document.getElementById("app"),
      );
```

Class component

```
<!doctype html>
<html>
  <head>
    <title>React Applicaction</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app"></div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script type="text/babel">
      class MyComponent extends React.Component {
        render() {
          return (
            <span>
              I am
              <em>
                {" "}
                {this.props.name} {this.props.lastname}
              </em>{" "}
              the {this.props.job}
            </span>
          );
        }
      }
      MyComponent.defaultProps = {
        job: "Engineer",
      };
      ReactDOM.render(
        <MyComponent name="Titor" lastname="Oopart" />,
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

Nota como el metodo render() reotrna el estado envolviendo el valore retornado en parentesis. Esto es por el mecanismo JavaScript insercion automatica de semi-colon(ASI). Un estado retorno seguido por una nueva linea en el mismo return; Cual es el mismo return undefined; lo cual es lo que no quieres.
Envolver la expression retorno en parentesis termite mejor formato de codigo.

4.8. State

El ejemplo sera statico(o "stateless"). El objetivo fue solo darte una idea de construir bloques de compocicion para UI. Pero React se complica cuando viejos navegadores manipulan DOM y el mantenimiento se hace complicado es cuando el dato en tu applicacion cambia. React tiene el concepto de state, cual es cualquier dato que el componente quiere para usar el render por ellos mismos.
Cuando el estado cambia React reconstruye la UI en el Dom sin que tengas que hacer nada. Despues construyes tu UI inicial en tu metodo render() o en la funcion rendering en caso de una funcion component todo lo que debes preocuparte es actaulzar la data. No necesitas preocuparte sobre cambios UI del todo. Despues de todo tu metodo/funcion render esta lista proveyendo el blueprint de como el deberia verse el componente
Similar a como tus propiedades de acceso via this.props tu lees el estado via el objeto this.state Para actualizar el estado, Tu usas this.setState() Cuando this.setState() es llamado React llama el metodo render de tu componente y todos sus hijos y actualiza la UI La actualizacion de la UI despues de llamar this.setState() esta hecha usando un mecanismo que eficientemente maneja cambios. Actualizado this.state directamente puede tener un comportamiento inesperado y tu no deberias hacerlo.
Como con this.props considera el this.state object read-only no solo por que es semanticamente una idea mala peor por que esto puede actuar en maneras tu no esperas. Similar no llames this.render() a ti mismo deja a React manejar los cambios, llama render() cuando y si es apropiado.

4.9. Textarea Component

Construyamos un nuevo componente textarea que cuenta el numero de caracteres escritos Tu (o bien otro futuro consumidor de este asombroso componente reusable) puede usar el nuevo component como
Crearemos una version "stateless"que no maneje los updates; Esto no es diferente a los ejemplos anteriores

```
<!doctype html>
<html>
  <head>
    <title>React Application stateless</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app"></div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script type="text/babel">
      class TextAreaCounter extends React.Component {
        render() {
          const text = this.props.text;
          return (
            <div>
              <textarea defaultValue={text} />
              <h3>{text.length}</h3>
            </div>
          );
```

```
        }
      }
      TextAreaCounter.defulatProps = {
        text: "Count me as I type",
      };

      ReactDOM.render(
        <TextAreaCounter text="Prueba" />,
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

Notaras <textarea>toma el la propiedad defaultValue como opuesto al texto que escribiamos en el nodo hijo. Esto es por que hay algunos diferencias entre React y old-school HTML cuando hacemos form elements. Lo discutiremos mas adelante, Adicionalmente encontraras estas difrencias haran tu vida de desarrollador mas facil

4.10. Hacerlo Stateful

Ahora hagamos que el componente stateless sera un stateful. Basicamente actualizara su estado cada ves que sea actualizado Primero necesitas setear el estado inicial en la clase constructor usando this.state Ten en mente que el constructor es el unico lugar donde esta bien setear el estado directamente sin llamar this.setState().
Es requerido iniciar this.state si no lo haces accessos consecutivos a this.state en el metodo render() fallaran
En este caso no es necesario inicializar this.state.text con un valor como puedes ver a la paropiedad this.prop.text
El dato de este componente es el contenido del textarea, entonce el estado tiene solo una propiedad llamada text, cual es accessible via this.state.text despues necesitas actualizar el estado puedes usar un metodo helper para este proposito

```
        onTextChange(event) {
          this.setState({
            text: event.target.value,
          });
        }
```

Siempre actualizaras el estado con this.setState(), cual toma un objeto y une con el dato existente en this.state. Como onTextChange() es un evento manejado que toma un evento y crece dentro para obtener el contenido del input textarea
La ultima cosa es actualziar el metodo render render() configurando el event handler

```
        render() {
          const text = "text" in this.state ? this.state.text : this.
    props.text;
          return (
            <div>
              <textarea
                value={text}
                onChange={(event) => this.onTextChange(event)}
              />
              <h3>{text.length}</h3>
```

```
            </div>
          );
        }
```

Ahora donde sea que el usuario escriba en el textarea el contador de caracteres actualizara para reflejar el contenido
En este ejemplo implementamos onChange significa que textarea es ahora controllado por React hablaremos mas de controlled components mas adelante.

4.11. Una nota sobre DOM Events

Para evitar confuciones, una pequenia clarificacion epara recordar la siguiente linea:

```
onChange={event => this.onTextChange(event)}
```

React usa su propio sistema de eventos synthetic para rendimiento(como bien es conveniente y saludable) Para entender por que necesitas considerar como estan hechas las cosas en el mundo puro DOM

4.12. Event Manejado en los viejos tiempos

Es conveniente usar manejador de eventos inline para hacer cosas como esta

```
<button onclick="doStuff">
```

Mientras e conveniente y facil de leer(el event listener es correcto con el codigo de la UI) Es ineficiente tener muchos event listener como estos Es dificil tener mas que un listener en el mismo button, especialmente si dicho button esta en algunlado mas component o libreri y tu no quiere ir ahi y arreglarlo o hacer fork el codigo.
Esto es por que el mundo DOM es comun usar element.addEventListener para configurar listeners(con cual ahora manejamos para tener un codigo en dos lugares o mas) una delegacion evento(para direccionar los problemas de rendimiento)
Event delegation significa que tu escuchas a eventos en algun lugar del nodo padre. un que contiene muchos buttons y tu configuras un listener para todos los buttons, en lugar de un listener para cada button. Entonces le dejas el manejo de evento a una autoridad paddre

```
<!doctype html>
<html>
  <head>
    <title>React Application button handler</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="parent">
      <button id="ok">OK</button>
      <button id="cancel">Cancel</button>
    </div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script>
      document
```

```
        .getElementById("parent")
        .addEventListener("click", function (event) {
          const button = event.target;

          // Hace cosas diferentes basadas en que boton fue clickeado
          switch (button.id) {
            case "ok":
              console.log("OK!");
              break;
            case "cancel":
              console.log("Cancel");
              break;
            default:
              new Error("Unexpected button ID");
          }
        });
    </script>
  </body>
</html>
```

Esto funciona y se lleva a cabo bien, pero hay drawbacks

- Declarando el listener es mejro manera del componente UI, cual hace codigo dificil de encontrar y debugear
- Usando delegacion y simpre switch-ing creamos unnecesarios codigo builerplate incluso antes de hacer el trabajo(respondiendo al button click en este caso)
- Inconcistencia de navegacion (omitido aqui) actualmente requiere este codigo para que sea largo

Desafortunadamente cuando esto viene tomandoesto codigo vive en el frente de usuarios reales necesitas un poco mas de adiciones si quieres que soporte navegadores antiguos

- Necesitas attachEvent adicionalmente para addEventListener
- Necesitas const event = event || window.event; arriba del listener
- Necesitas const button = event.target || event.srcElement;

Todo esto es necesario y hace suficiente que termines usando una libreria evento de algun orden. Pero porque agregar otra libreria(y estudiando mas APIs)Cuando React viene bundled con una solucion al event-handling nightmares?

4.13. Manejo de Eventos en React

React usa eventos synteticos para envolver y normalizar los eventos del navegador. lo cual significa no habra inconsistencias en el navegador
La sintasis hace hacil mantener la UI y el event listener juntos. React usa event delegation por razones de rendimientos
React usa sintaxis camelCase para el evetnt handler entonces usamo onClick en lugar de onclick
Si por alguna razon necesitas el evento original del browser esta disponible como event.nativeEvent
El evento onChange(como se uso en el ejemplo textarea)comporta como esperas. Esto se enciendo cuando el usuario escibe opuesta a despues de que el escriba de escribir y haya navegado en el campo, lo cual es un comportamiento en DOM

4.14. Sintaxis de manejo de Event

En este ejemplo se usa una funcion =>para llamar el helper onTextChange

```
onChange={event => this.onTextChange(event)}
```

Esto es porque el onChange=this.onTextChange no funcionara
Otra opcion es bind el metodo como esto

```
onChange={this.onTextChange.bind(this)}
```

```
<!doctype html>
<html>
  <head>
    <title>React Application button handler</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app"></div>
    <div id="parent">
      <button id="ok">OK</button>
      <button id="cancel">Cancel</button>
    </div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script type="text/babel">
      class MyComponent extends React.Component {
        constructor() {
          super();
          this.state = {};
        }
        conuntText(event) {
          this.setState({ text: event.target.value });
          console.log(event.target.defaultValue);
        }
        render() {
          const text = "text" in this.state ? this.state.text : this.
    props.text;
          return (
            <div>
              <span>{this.props.name}</span>
              <br />
              <textarea onChange={this.conuntText.bind(this)} value={
    text} />
              <h3>{text.length}</h3>
            </div>
          );
        }
      }
      MyComponent.defaultProps = {
        text: "Default",
      };
      ReactDOM.render(
        <MyComponent name="Titor" />,
        document.getElementById("app"),
      );
    </script>
  </body>
```

```
</html>
```

Y otra opcion y un patron comun es bind todos los eventos manejado en el contructor

```
      class MyComponent extends React.Component {
        constructor() {
          super();
          this.state = {};
          this.conuntText = this.conuntText.bind(this);
        }
        conuntText(event) {
          this.setState({ text: event.target.value });
          console.log(event.target.defaultValue);
        }
        render() {
          const text = "text" in this.state ? this.state.text : this.
    props.text;
          return (
            <div>
              <span>{this.props.name}</span>
              <br />
              <textarea onChange={this.conuntText} value={text} />
```

un poco necesario boilerplate, pero esta maenra el manejo de eventos se solo una ves opuesto a hacerlo cada vez llamado el metodo render() lo cual ayuda reducir el uso de la memoria en tu aplicacion

Este patron comun fue en gran parte reemplazada una ves es posible usar funciones como propiedades de clase en javascript

```
// Antes
class TextAreaCounter extends React.Component {
  constructor (){
    super();
    this.state = {}
    this.myMethodEvent = this.myMethodEvent.bind(this);
  }
  myMethodEvent(event) {
    this.setState({
      .....
    })

  }
}
// Despues
class TextAreaCounter extends React.Component {
  constructor() {
    super();
    ....
  }
  changeText = (event) => {
    ....
  }
}
```

```
      class MyComponent extends React.Component {
        constructor() {
          super();
          this.state = {};
        }
```

```
        conuntText = (event) => {
          this.setState({ text: event.target.value });
          console.log(event.target.defaultValue);
        };
        render() {
          const text = "text" in this.state ? this.state.text : this.
    props.text;
          return (
            <div>
              <span>{this.props.name}</span>
              <br />
              <textarea onChange={this.conuntText} value={text} />
              <h3>{text.length}</h3>
            </div>
          );
        }
      }
```

4.15. Props Versus State

Ahora que conoces que tienes que acceder a this.props y this.state cuando cuando vienes a mostrar tus compoenntes en tu metodo render(). Ahora debes saber cuando usar el uno o el otro
Las propiedades son mecanismos para el mundo de afuera para configurar tu componente. State es tu mantenmiento de datos internos. Entonces si tu consideras una analogia con OOP, this.props es como una collection de todos los argumentos pasados a la clase constructor, mientras this.state es una mochila de tus propiedades privadas
En general, prefiere separar tu aplicacion en una manera que tu tienes pocos stateful components y mas stateless components

4.16. Props en estado inicial: Un Antipatron

```
// Peligro esto es un Antipatron
  this.state = {
    text: props.text,
  }
```

Esto es considrado un antipagron. Idealmente usas una conbinacion de this.state y this.props y como ves para construirlo en tu metodo render(). Pero algunas veces tu quieres tomar un valor pasado a tu componente y usarlo para construir el estado inicial. No hay nada de malo con esto excepto que las llamadas de tu componente puede esperar una propiedad(texto en el ejemplo) para siempre tener el ultimo valor, y el codigo violira este comportamiento esprado. PAra setear la expectativa directa, un simple cambio de nombramiento es suficiente por ejemplo llamado la propiedad algo como defaultText o initialValue en lugar de solo text

4.17. Accediendo al Componente desde afuera

Tu no siempre tienes el lujo de empezar una marca nueva React app desde cero Algunas veces necesitas un gancho dentro una applicacion existente o una website y migrarlo a React una piesa a la vez. Afortunadamente React fue diseniado para trabajar con cualquier codigobase preexistente que tengas. Despues de todo, el creador original de React no puede detener el mundo y reescribir una aplicacion entera(Facebook.com) desde 0 especialmente en el tempranos didas cuando React era joven.
Una manera que tu aplicacion React puede comunicarse con el mundo de afuera es para tener una ferencia de un componente tu renderisas con ReactDOM.render() y los usas desde afuera del componente

```
const myTextAreaCounter = ReactDom.render(
  <TextAreaCounter text="Bob" />,
  document.getElementById("app")

)
```

Ahora puedes usar myTextAreaCounter para acceder al mismo metodo y propiedades que normalmente accedias con esto cuando dentro del componente. Tu puedes incluso jugar con los componentes usando tu consola JavaScript

```
      const myTextAreaCounter = ReactDOM.render(
        <MyComponent text="Bob" />,
        document.getElementById("app"),
      );
```

En este ejemplo myTextAreaCounter.state revisa el estado actual(inicialmente vacio); myTextAreaCounter.props revisa las propiedades y esta linea setea un nuevo estado

```
myTextAreaCounter.setState({text: "Hello outside world!"});
```

Esta linea obtiene una referencia al padre nodo principal DOM que React crea

```
const reactAppNode = ReactDOM.findDOMNode(myTextAreaCounter)
```

4.18. Ciclo de vida Methodos

React ofrese muchos mmetodos llamados lifecycle. Puedes usar el metodos lifecycle para escuchar cambios tus componentes
La vida de un componentes viene atravez de tres pasos:

- Mounting, El componente es agregado al DOM inicialmente
- Updating, El componente es actualizado como un resultado del llamado setState() y/o proyendo prop al componente cambiado
- Unmounting, El componente es removido del DOM

React se hace parte de este trabajao antes actualizar el DOM. Esto es tambien llamado la fase de renderisado. Despues esto actualiza el DOM y elsta fase es llamado una fase commit Con el background considremos algunso lifecycle methods

- Despues del montaje inicial y despues de commit el DOM, tus comopnentes metodo componentDidMount() es llamado, si existe. Este es el lugar para hacer cualquier inicializacion funcione como requiere el DOM. Cualquier inicializacion funciona que no requiere el DOM debe estar en el contructor.
  Y mas de tu inicializacion no deberia requerir el DOM. Pero en este metodo puedes por ejemplo asegurarte el altura del componente que fue solo renderizado, agregar cualquier event listener addEventListener('resize') u obtener los cambios del servidor.

- Antes deque el componente es removido del DOM, el metodo componentWillUnmount() es llamado. Este es el lugar para hacer un clean que necesitaras. Cualquier evento manejado, o cualquier cosa que puede filtrar memorya, deberia ser limpiada, despues de esto el componente se fue para siempre

- componentDidUpdate(previousProps, previousState, snapshot) esto es llamado donde sea el componente fue actualizado. Desde este punto this.props y this.state tienen actualizado valores, tendras una copia previa. Puedes usar esta informacion para conparar el viejo y el nuevo estado y potencialmente hacer mas request a la red si es necesario.

- Y despues hay shouldComnponentUpdate(newProps, newState) cual es una oportunidad para una optimizacion. Estas dando el estado que sera cual puedes comporar casos si el metodo render() no es llamado

De estos componentDidMount() y componentDidUpdadte() son los mas comunes

4.19. lifecycle ejemplo: Log it All

Para un mejor entendimiento la vida de un componente, agreguemos algunos logging en el TextArea counter component. Simplemente implementando todo el metodo lifecycle para log a la consola cuando ellos estan invocados, juntos con ningun argumento:

```
class TextAreaCounter extends React.Component {
  constructor() {
    super();
    this.state = {};
  }
  countEvent = (event) => {
    this.setState({
      text: event.target.value,
    });
  };
  componentDidMount() {
    console.log("componentDidMount");
  }
  componentWillUnmount() {
    console.log("componentWillUnmount");
  }
  componentDidUpdate(prevProps, prevState, snapshot) {
    console.log("componentDidUpdate", prevProps, prevState);
    return "Hello";
  }
  shouldComponentUpdate(newProps, newState) {
```

```
22          console.log("shouldComponentUpdate", newProps, newState);
23          return true;
24        }
25        render() {
26          const text = "text" in this.state ? this.state.text : this.
    props.text;
27          return (
28            <div>
29              <textarea
30                onChange={this.countEvent.bind(this)}
31                defaultValue="Editing.."
32                value={text}
33              />
34              <h3>{text.length}</h3>
35            </div>
36          );
37        }
38      }
39      TextAreaCounter.defaultProp = {
40        text: "Default text",
41      };
42      const myTextAreaCounter = ReactDOM.render(
43        <MyComponent text="Bob" />,
44        document.getElementById("app"),
45      );
46      //      ReactDOM.render(<TextAreaCounter />, document.
    getElementById("app"));
```

Despues de cargar la pagina, el unico mensaje que sale la consola es componentDidMount Luego cuando escribes algo en el textarea se llama al metodo shouldComponentUpdate() con los nuevos props(iguales al anterior) y el nuevo estado.
Desde este metodo retorna true, React procede llamado getSnapShotBeforeUpdate() pasando las viejas props y state. Esto es tu chance para hcaer algo con ellos y con el viejo DOM y pasar cualquier informacion como un snapshot al siguiente metodo.
Por ejemplo esto es una oportunidad para hacer algunas medidas de elemento o un posicion scroll y snapshot despues ves si ellos cambiaron despues del update. Finalmente componentDidUpdate() es llamado con la vieja informacion(tienes el nuevo en this.state y this.props) y cualquier snapshot definido por el metodo previo Finalmente para demostrar componentWillUnmount() en accion (usando el ejemplo) puedes escribir en la consola

```
1 ReactDOM.render(React.createElement('p', null, 'Enough counting!'), app
    );
```

Esto remplaza todo el textare component con un nuevo <p>component. Despues podras ver el mensaje componentWillUnmount en la consola

4.20. Paranoid State Protection

Decir que tu quieres restringir el numero de caracteres a ser escrito en el textare. Podrias hacer esto en el event handler onTextChange(), cual es llamar como el usertypes Pero que si alguien llama setState() desde fuera del componente (como mensione antes, es una mala idea?) puedes todavia estar protegido la inconsitencia y haciendo buen componente? Seguro. Puedes hacer la validacion en componentDidUpdate() y si el numero

de caracteres es mayor que el permitido, revierte el estado algo como esto.

```
componentDidUpdate(prevPros, prevState) {
  if (this.satte.text.length > 3) {
    this.setState({
      text: prevState.text || this.props.text
    })
  }
  }
```

```
<!doctype html>
<html>
  <head>
    <title>React Application</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <div id="app"></div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script type="text/babel">
      class TextAreaCounter extends React.Component {
        constructor() {
          super();
          this.state = {};
          this.countEvent = this.countEvent.bind(this);
        }
        countEvent = (event) => {
          this.setState({
            text: event.target.value,
          });
        };
        componentDidMount() {
          console.log("componentDidMount");
        }
        componentWillUnmount() {
          console.log("componentWillUnmount");
        }
        /*
        componentDidUpdate(prevProps, prevState, snapshot) {
          console.log("componentDidUpdate", prevProps, prevState);
          if (this.state.text.length > 3) {
            this.setState({
              text: prevState.text || this.props.text,
            });
          }
          return "Hello";
        }
        */
        shouldComponentUpdate(_, newState) {
          console.log("shouldComponentUpdate", newState);
          return newState.text.length > 3 ? false : true;
        }
        render() {
          const text = "text" in this.state ? this.state.text : this.
    props.text;
          return (
            <div>
```

```
              <textarea onChange={this.countEvent} value={text} />
              <h3>{text.length}</h3>
            </div>
          );
        }
      }
      TextAreaCounter.defaultProp = {
        text: "Default text",
      };
      const myTextAreaCounter = ReactDOM.render(
        <TextAreaCounter text="Bob" />,
        document.getElementById("app"),
      );
      //      ReactDOM.render(<TextAreaCounter />, document.
    getElementById("app"));
    </script>
  </body>
</html>
```

4.21. lifecycle ejemplo: Usando un componente hijo

Tu sabes que puedes juntar y anidar React components como quieras. Tienes solo tienes que ver ReactDOM components en el metodo render() hechemos un vistaso a otro simple custom component a ser usado como un hijo
Aisla la parte responsable para el ocunter dentor su propio componente despues despues todo, divide y conquista lo es todo.
Primero isolemos el lifestyle loggin dentro una clase separada y tendras los components heredados .
Heerencia es nunca garantisado cuando viene a React porque para el trabajo UI, composicion es preferible y por nonUI funcione. un modulo regular javascript podria hacerlo. Todavia es util saber como funciona, y esto te ayuda a evitar copy-pasting el metodo logging
Este es el padre:

```
      class LifecycleLoggerComponent extends React.Component {
        static getName() {}
        componentDidMount() {
          console.log(this.constructor.getName() + "::componentDidMount
    ");
        }
        componentWillUnmount() {
          console.log(this.constructor.getName() + "::
    componentWillUnmount");
        }
        componentDidUpdate(prevProps, prevState, snapshot) {
          console.log(this.constructor.getName() + "::
    componentDidUpdate");
        }
      }
```

El nuevo componente count siempre muestra count no mantiene el estado pero muestra la propiedad count dato por los padres

```
      class Counter extends LifecycleLoggerComponent {
        static getName() {
```

```
          return "Counter";
        }
        render() {
          return <h3>{this.props.count}</h3>;
        }
      }
      Counter.defaultProps = {
        count: 0,
      };
```

El component textarea configura un static metodo getName()

```
class TextAreaCounter extends LifecycleLoggerComponent {
  static getName() {
    return 'TextAreaCounter';
  }
  }
```

Y finalmente el textarea render() obtendra el uso de <Counter />y lo usara condicionalmente; si el count es 0, no mostrara nada

```
         render() {
          const text = "text" in this.state ? this.state.text : this.
    props.text;
          return (
            <div>
              <textarea value={text} onChange={this.onTextChange} />
              {text.length > 0 ? <Counter count={text.length} /> : null
    }
            </div>
          );
        }
      }
```

4.22. Rendimiento Win: Prevenit Actualizacion de componentes

Ya conoces sobre shouldComponentUpdate() y lo viste en accion. Es especialmente importante cuando construimos rendimiento-critico para tu aplicacion Esta invocando antes componentWillUpdate() y da chance de cancelar el update si asi lo decides
Hay una clase de componentes que usa solo this.props y this.state en su metodos render() y no funciones adicionales a llamar. Estos compoenntes son llamados componentes puros Ellos pueden implementar shouldComponentUpdate() y compara el estado y las propiedades antes y despues de un update y si no hay cambios retornan false y guardan alguna energia de procesamiento. ellos pueden ser pure static components que usa neither props nor state. Estos pueden asegurar retornar falso
React puede hacerlo facil revisando todos los props y states en shouldComponentUpdate() en lugar de repetir este trabajo puedes tener tus componentes heredados React.PureComponent y que los metodos getName sean static de esta manera no necesitaras implementar shouldComponentUpdate() Esta hecho para ti.
Desde heredamos ambos componentes el logger todo lo que necesitas es

```
<!doctype html>
<html>
  <head>
```

```
4     <meta charset="utf-8" />
5     <title>React Practice child component</title>
6   </head>
7   <body>
8     <div id="app"></div>
9     <script src="../reactbook/react/react.js"></script>
10    <script src="../reactbook/react/react-dom.js"></script>
11    <script src="../reactbook/react/babel.js"></script>
12    <script type="text/babel">
13      class MyParentComponent extends React.PureComponent {
14        static getName() {}
15        componentDidUpdate() {
16          console.log(this.constructor.getName() + "::
    componentDidUpdate");
17        }
18        componentDidMount() {
19          console.log(this.constructor.getName() + "::componentDidMount
    ");
20        }
21        componentWillUnmount() {
22          console.log(this.constructor.getName() + "::
    componentWillUnmount");
23        }
24      }
25      class Counter extends MyParentComponent {
26        static getName() {
27          return "Counter";
28        }
29        render() {
30          console.log(this.constructor.getName() + " :: render");
31          return <h3>{this.props.count}</h3>;
32        }
33      }
34      Counter.defaultProps = {
35        count: 0,
36      };
37
38      class TextBoxCounter extends MyParentComponent {
39        static getName() {
40          return "TextBoxCounter";
41        }
42        constructor() {
43          super();
44          this.state = {};
45          this.textCapture = this.textCapture.bind(this);
46        }
47        textCapture(event) {
48          this.setState({
49            text: event.target.value,
50          });
51        }
52        render() {
53          console.log(this.constructor.getName() + " :: render");
54          const text = "text" in this.state ? this.state.text : this.
    props.text;
55          return (
56            <div>
57              {" "}
```

```
              <textarea onChange={this.textCapture} value={text} />
              {text.length > 0 ? <Counter count={text.length} /> : null
    }
            </div>
          );
        }
      }
      TextBoxCounter.defaultProps = {
        text: "Default",
      };
      const MyRenderComponent = ReactDOM.render(
        <TextBoxCounter />,
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

Notaras que cargando la pagina los componentes se renderisan y montan
TextAreaCounter::render
Counter::render
Counter::componentDidMount
TextAreaCounter::componentDidMount
Haciendo cambios aumentando texto los componentes se renderizaran y actualizaran
TextAreaCounter::render Counter::render Counter::componentDidUpdate TextAreaCounter::componentDidUpdate
Y si remplasas el texto de el texbox con otra palabra de la misma cantidad de caracteres
TextAreaCounter::render
TextAreaCounter::componentDidUpdate
Veras que el re-render Counter ya no es ejecutado por que la cantidad de caracteres sigue siendo la misma

Capítulo 5

Excell Una fantastica Tabla de Componentes

Ahora que sabes como crear custom React Componentes y muchas cosas mas. Pongamos todo esto a prueba creando una tabla de datos

5.1. Primero los Datos

Aqui tenemos un ejemplo

```
const headers = ['Book', 'Author', 'Language', 'Published', 'Sales'];
const data = [
[
'A Tale of Two Cities', 'Charles Dickens',
'English', '1859', '200 million',
],
[
'Le Petit Prince (The Little Prince)', 'Antoine de Saint-Exupéry',
'French', '1943', '150 million',
],
[
"Harry Potter and the Philosopher's Stone", 'J. K. Rowling',
'English', '1997', '120 million',
],
[
'And Then There Were None', 'Agatha Christie',
'English', '1939', '100 million',
],
[
'Dream of the Red Chamber', 'Cao Xueqin',
'Chinese', '1791', '100 million',
],
[
'The Hobbit', 'J. R. R. Tolkien',
'English', '1937', '100 million',
],
];
```

5.2. Table Headers Loop

Creamos el iterador para contruir la cabecera

```
<!doctype html>
<html>
  <header>
    <meta charset="utf-8" />
    <title>Header Loop Table</title>
    <link rel="stylesheet" type="text/css" href="03.table.css" />
  </header>
  <body>
    <div id="app"></div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script type="text/babel">
      const headers = ["Book", "Author", "Language", "Published", "
    Sales"];
      const data = [
        [
          "A Tale of Two Cities",
          "Charles Dickens",
          "English",
          "1859",
          "200 million",
        ],
        [
          "Le Petit Prince (The Little Prince)",
          "Antoine de Saint-Exupéry",
          "French",
          "1943",
          "150 million",
        ],
        [
          "Harry Potter and the Philosopher's Stone",
          "J. K. Rowling",
          "English",
          "1997",
          "120 million",
        ],
        [
          "And Then There Were None",
          "Agatha Christie",
          "English",
          "1939",
          "100 million",
        ],
        [
          "Dream of the Red Chamber",
          "Cao Xueqin",
          "Chinese",
          "1791",
          "100 million",
        ],
        ["The Hobbit", "J. R. R. Tolkien", "English", "1937", "100
    million"],
      ];
```

```
      class Excell extends React.Component {
        render() {
          const headers = [];
          for (const title of this.props.headers) {
            headers.push(<th>{title}</th>);
          }
          return (
            <table>
              <thead>
                <tr>{headers}</tr>
              </thead>
            </table>
          );
        }
      }
      ReactDOM.render(
        <Excell headers={headers} />,
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

Como puedes ver renderisamos el array headers en tr el cual fue compuesto por el metodo for

5.3. Agregando Content < td >

Ahora que tiene un cabecera de tabla bien es hora de agregar el body, ahora renderisaremos un array de dos dimenciones filas y columnas
Para pasar el dato a Excel usaremo un nuevo prop llamado initialData. Porque initial y no lo sata? Es por manejo esperado. El usuario podra editarlo ordenarlo y un poco mas, en otras palabras el estado del componente cambiara Entonces usemos this.state.data para mantener el trackeo de los cambios y usar this.props.initialData

```
<!doctype html>
<html>
  <head>
    <title>Sort Table</title>
    <meta charset="utf-8" />
    <link rel="stylesheet" type="text/css" href="03.table.css" />
  </head>
  <body>
    <div id="app"></div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script type="text/babel">
      const headers = ["Book", "Author", "Language", "Published", "
    Sales"];
      const data = [
        [
          "A Tale of Two Cities",
          "Charles Dickens",
          "English",
          "1859",
```

```
21          "200 million",
22        ],
23        [
24          "Le Petit Prince (The Little Prince)",
25          "Antoine de Saint-Exupéry",
26          "French",
27          "1943",
28          "150 million",
29        ],
30        [
31          "Harry Potter and the Philosopher's Stone",
32          "J. K. Rowling",
33          "English",
34          "1997",
35          "120 million",
36        ],
37        [
38          "And Then There Were None",
39          "Agatha Christie",
40          "English",
41          "1939",
42          "100 million",
43        ],
44        [
45          "Dream of the Red Chamber",
46          "Cao Xueqin",
47          "Chinese",
48          "1791",
49          "100 million",
50        ],
51        ["The Hobbit", "J. R. R. Tolkien", "English", "1937", "100
    million"],
52      ];
53 ....
54      class Excell extends React.Component {
55        constructor(props) {
56          super();
57          this.state = { data: props.initialData };
58        }
59        render() {
60          const headers = [];
61          for (const idx in this.props.headers) {
62            const header = this.props.headers[idx];
63            headers.push(<th>{header}</th>);
64          }
65          return (
66            <div>
67              <h3>Table</h3>
68              <table>
69                <thead>
70                  <tr>{headers}</tr>
71                </thead>
72                <tbody>
73                  {this.state.data.map((row, idx) => (
74                    <tr key={idx}>
75                      {row.map((cell, idx) => (
76                        <td key={idx}>{cell}</td>
77                      ))}
```

```
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          );
        }
      }
      ReactDOM.render(
        <Excell headers={headers} initialData={data} />,
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

En este caso se usara un doble map para hacer el recorrido de cada celda en cada fila y columna tambien podria usarse un doble for anidado pero no seria buena practica

5.4. Prop Types

La habilidad para especificar tipos de variables como (string, numeros, booleans etc) no existe en Javacript. Pero los desarrolladores de otros lenguajes, y estos que trabajan en proyecto grandescon muchos otros desarrolladores, lo olvidan.
Dos opciones populares existe nque permiten escribir JavaScript con tipos: Flow y TypeScript. Puedes esar estos para escribir applicaciones React.
Pero otra opcion existente cual es limitada para solo especificar el tipo de props en tu componente con prop types. Ellos fueron parte de React, inicialmente pero fueron movidos a una librearia separada en React v15.5
Prop types te permite ser mas especifico como que dato de la tabla tomar y como un resultado un error en el developer console aparecera. Puedes configurar el prop type como esto
Para esto necesitaras una nueva libreria

```
curl -L https://unpkg.com/prop-types/prop-types.js > ~/reactbook/react/
    prop-types.js
```

y especificar lo siguiente

```
<!doctype html>
<html>
  <head>
    <title>Sort Table</title>
    <meta charset="utf-8" />
    <link rel="stylesheet" type="text/css" href="03.table.css" />
  </head>
  <body>
    <div id="app"></div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script src="react/prop-types.js"></script>
    <script type="text/babel">
      const headers = ["Book", "Author", "Language", "Published", "
    Sales"];
      const data = [
```

```
17          [
18            "A Tale of Two Cities",
19            "Charles Dickens",
20            "English",
21            "1859",
22            "200 million",
23          ],
24          [
25            "Le Petit Prince (The Little Prince)",
26            "Antoine de Saint-Exupéry",
27            "French",
28            "1943",
29            "150 million",
30          ],
31          [
32            "Harry Potter and the Philosopher's Stone",
33            "J. K. Rowling",
34            "English",
35            "1997",
36            "120 million",
37          ],
38          [
39            "And Then There Were None",
40            "Agatha Christie",
41            "English",
42            "1939",
43            "100 million",
44          ],
45          [
46            "Dream of the Red Chamber",
47            "Cao Xueqin",
48            "Chinese",
49            "1791",
50            "100 million",
51          ],
52          ["The Hobbit", "J. R. R. Tolkien", "English", "1937", "100
    million"],
53        ];
54 ....
55        class Excell extends React.Component {
56          constructor(props) {
57            super();
58            this.state = { data: props.initialData };
59          }
60          render() {
61            const headers = [];
62            for (const idx in this.props.headers) {
63              const header = this.props.headers[idx];
64              headers.push(<th key={idx}>{header}</th>);
65            }
66            return (
67              <div>
68                <h3>Table</h3>
69                <table>
70                  <thead>
71                    <tr>{headers}</tr>
72                  </thead>
73                  <tbody>
```

```
                    {this.state.data.map((row, idx) => (
                      <tr key={idx}>
                        {row.map((cell, idx) => (
                          <td key={idx}>{cell}</td>
                        ))}
                      </tr>
                    ))}
                  </tbody>
                </table>
              </div>
            );
          }
        }
        Excell.propTypes = {
          header: PropTypes.arrayOf(PropTypes.string),
          initialData: PropTypes.arrayOf(PropTypes.arrayOf(PropTypes.
    string)),
        };
        ReactDOM.render(
          <Excell headers={headers} initialData={data} />,
          document.getElementById("app"),
        );
      </script>
    </body>
</html>
```

5.5. Sorting

Cuantas veces viste una tabla en la que puedes ordenarla diferente? Es trivial hacer esto con React. Acualmente este es un ejemplo donde React porque todo lo que necesitas hacer es ordenar el array y todo la UI lo actualizara por ti
Por conveniencia y leible todos los metodos de ordenamiento estan en un metodo sort()

```
<!doctype html>
<html>
  <head>
    <title>Sort Table</title>
    <meta charset="utf-8" />
    <link rel="stylesheet" type="text/css" href="03.table.css" />
  </head>
  <body>
    <div id="app"></div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script src="react/prop-types.js"></script>
    <script type="text/babel">
      const headers = ["Book", "Author", "Language", "Published", "
    Sales"];
      const data = [
        [
          "A Tale of Two Cities",
          "Charles Dickens",
          "English",
          "1859",
          "200 million",
```

```
23          ],
24          [
25            "Le Petit Prince (The Little Prince)",
26            "Antoine de Saint-Exupéry",
27            "French",
28            "1943",
29            "150 million",
30          ],
31          [
32            "Harry Potter and the Philosopher's Stone",
33            "J. K. Rowling",
34            "English",
35            "1997",
36            "120 million",
37          ],
38          [
39            "And Then There Were None",
40            "Agatha Christie",
41            "English",
42            "1939",
43            "100 million",
44          ],
45          [
46            "Dream of the Red Chamber",
47            "Cao Xueqin",
48            "Chinese",
49            "1791",
50            "100 million",
51          ],
52          ["The Hobbit", "J. R. R. Tolkien", "English", "1937", "100
    million"],
53        ];
54        function clone(o) {
55          return JSON.parse(JSON.stringify(o));
56        }
57        class Excell extends React.Component {
58          constructor(props) {
59            super();
60            this.state = { data: props.initialData };
61            this.sort = this.sort.bind(this);
62          }
63          sort(e) {
64            const column = e.target.cellIndex;
65            console.log(column, e.target);
66            const data = clone(this.state.data);
67            data.sort((a, b) => {
68              if (a[column] === b[column]) {
69                return 0;
70              }
71              return a[column] > b[column] ? 1 : -1;
72            });
73            this.setState({ data });
74          }
75          render() {
76            const headers = [];
77            for (const idx in this.props.headers) {
78              const header = this.props.headers[idx];
79              headers.push(<th key={idx}>{header}</th>);
```

```
          }
          return (
            <div>
              <h3>Table</h3>
              <table>
                <thead onClick={this.sort}>
                  <tr>{headers}</tr>
                </thead>
                <tbody>
                  {this.state.data.map((row, idx) => (
                    <tr key={idx}>
                      {row.map((cell, idx) => (
                        <td key={idx}>{cell}</td>
                      ))}
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          );
        }
      }
      Excell.propTypes = {
        headers: PropTypes.arrayOf(PropTypes.string),
        initialData: PropTypes.arrayOf(PropTypes.arrayOf(PropTypes.
    string)),
      };
      ReactDOM.render(
        <Excell headers={headers} initialData={data} />,
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

Aqui usamos el metodo sort el cual puede desmenusar el array en y empezara comprar una fila con la siguiente y devolver 1, -1, 0 si se necesita ordenarlo

5.6. Sorting UI Cues

La tabla es ordenada sin problemas pero no es claro que columna fue ordenada, actualicemos la UI para mostrar una flecha basada en la column Para mantener el tracking de los nuevos estados necesitaremos nuevos estados
this.state.sortby, Nombre de la columna que esta actualmente ordenando la tabla this.state.descending, True or False si esta ordenada ascendiente o descendiente

```
<!doctype html>
<html>
  <head>
    <title>Sort Table</title>
    <meta charset="utf-8" />
    <link rel="stylesheet" type="text/css" href="03.table.css" />
  </head>
  <body>
    <div id="app"></div>
    <script src="react/react.js"></script>
```

```
11     <script src="react/react-dom.js"></script>
12     <script src="react/babel.js"></script>
13     <script src="react/prop-types.js"></script>
14     <script type="text/babel">
15       const headers = ["Book", "Author", "Language", "Published", "
    Sales"];
16       const data = [
17         [
18           "A Tale of Two Cities",
19           "Charles Dickens",
20           "English",
21           "1859",
22           "200 million",
23         ],
24         [
25           "Le Petit Prince (The Little Prince)",
26           "Antoine de Saint-Exupéry",
27           "French",
28           "1943",
29           "150 million",
30         ],
31         [
32           "Harry Potter and the Philosopher's Stone",
33           "J. K. Rowling",
34           "English",
35           "1997",
36           "120 million",
37         ],
38         [
39           "And Then There Were None",
40           "Agatha Christie",
41           "English",
42           "1939",
43           "100 million",
44         ],
45         [
46           "Dream of the Red Chamber",
47           "Cao Xueqin",
48           "Chinese",
49           "1791",
50           "100 million",
51         ],
52         ["The Hobbit", "J. R. R. Tolkien", "English", "1937", "100
    million"],
53       ];
54       function clone(o) {
55         return JSON.parse(JSON.stringify(o));
56       }
57       class Excell extends React.Component {
58         constructor(props) {
59           super();
60           this.state = {
61             data: props.initialData,
62             sortby: null,
63             descending: false,
64           };
65           this.sort = this.sort.bind(this);
66         }
```

```
67        sort(e) {
68          const column = e.target.cellIndex;
69          const data = clone(this.state.data);
70          const descending =
71            this.state.sortby == column && !this.state.descending;
72          data.sort((a, b) => {
73            if (a[column] === b[column]) {
74              return 0;
75            }
76            return descending
77              ? a[column] < b[column]
78                ? 1
79                : -1
80              : a[column] > b[column]
81                ? 1
82                : -1;
83          });
84          this.setState({
85            data,
86            sortby: column,
87            descending,
88          });
89        }
90        render() {
91          const headers = [];
92          for (const idx in this.props.headers) {
93            const header = this.props.headers[idx];
94            headers.push(<th key={idx}>{header}</th>);
95          }
96          return (
97            <div>
98              <h3>Table</h3>
99              <table>
100               <thead onClick={this.sort}>
101                 <tr>
102                   {this.props.headers.map((title, idx) => {
103                     if (this.state.sortby === idx) {
104                       title += this.state.descending ? " \u2191" : "
    \u2193";
105                     }
106                     return <th key={idx}>{title}</th>;
107                   })}
108                 </tr>
109               </thead>
110               <tbody>
111                 {this.state.data.map((row, idx) => (
112                   <tr key={idx}>
113                     {row.map((cell, idx) => (
114                       <td key={idx}>{cell}</td>
115                     ))}
116                   </tr>
117                 ))}
118               </tbody>
119             </table>
120           </div>
121         );
122       }
123     }
```

```
      Excell.propTypes = {
        headers: PropTypes.arrayOf(PropTypes.string),
        initialData: PropTypes.arrayOf(PropTypes.arrayOf(PropTypes.
    string)),
      };
      ReactDOM.render(
        <Excell headers={headers} initialData={data} />,
        document.getElementById("app"),
      );
    </script>
  </body>
</html>
```

5.7. Editable cell

Lo primero es configurar un event handler simple. En un doble click el componente sera renombrado

```
<!doctype html>
<html>
  <head>
    <title>Sort Table</title>
    <meta charset="utf-8" />
    <link rel="stylesheet" type="text/css" href="03.table.css" />
  </head>
  <body>
    <div id="app"></div>
    <script src="react/react.js"></script>
    <script src="react/react-dom.js"></script>
    <script src="react/babel.js"></script>
    <script src="react/prop-types.js"></script>
    <script type="text/babel">
      const headers = ["Book", "Author", "Language", "Published", "
    Sales"];
      const data = [
        [
          "A Tale of Two Cities",
          "Charles Dickens",
          "English",
          "1859",
          "200 million",
        ],
        [
          "Le Petit Prince (The Little Prince)",
          "Antoine de Saint-Exupéry",
          "French",
          "1943",
          "150 million",
        ],
        [
          "Harry Potter and the Philosopher's Stone",
          "J. K. Rowling",
          "English",
          "1997",
          "120 million",
        ],
```

```
      [
        "And Then There Were None",
        "Agatha Christie",
        "English",
        "1939",
        "100 million",
      ],
      [
        "Dream of the Red Chamber",
        "Cao Xueqin",
        "Chinese",
        "1791",
        "100 million",
      ],
      ["The Hobbit", "J. R. R. Tolkien", "English", "1937", "100 million"],
    ];
    function clone(o) {
      return JSON.parse(JSON.stringify(o));
    }
    class Excell extends React.Component {
      constructor(props) {
        super();
        this.state = {
          data: props.initialData,
          sortby: null,
          descending: false,
          edit: null,
        };
        this.sort = this.sort.bind(this);
        this.showEditor = this.showEditor.bind(this);
        this.save = this.save.bind(this);
      }
      sort(e) {
        const column = e.target.cellIndex;
        const data = clone(this.state.data);
        const descending =
          this.state.sortby == column && !this.state.descending;
        data.sort((a, b) => {
          if (a[column] === b[column]) {
            return 0;
          }
          return descending
            ? a[column] < b[column]
              ? 1
              : -1
            : a[column] > b[column]
              ? 1
              : -1;
        });
        this.setState({
          data,
          sortby: column,
          descending,
        });
      }
      save(e) {
        e.preventDefault();
```

```
95          const input = e.target.firstChild;
96          const data = clone(this.state.data);
97          data[this.state.edit.row][this.state.edit.column] = input.
    value;
98          this.setState({ edit: null, data });
99        }
100       showEditor(e) {
101         this.setState({
102           edit: {
103             row: parseInt(e.target.parentNode.dataset.row, 10),
104             column: e.target.cellIndex,
105           },
106         });
107       }
108       render() {
109         const headers = [];
110         for (const idx in this.props.headers) {
111           const header = this.props.headers[idx];
112           headers.push(<th key={idx}>{header}</th>);
113         }
114         return (
115           <div>
116             <h3>Table</h3>
117             <table>
118               <thead onClick={this.sort}>
119                 <tr>
120                   {this.props.headers.map((title, idx) => {
121                     if (this.state.sortby === idx) {
122                       title += this.state.descending ? " \u2191" : "
    \u2193";
123                     }
124                     return <th key={idx}>{title}</th>;
125                   })}
126                 </tr>
127               </thead>
128               <tbody onDoubleClick={this.showEditor}>
129                 {this.state.data.map((row, rowidx) => (
130                   <tr key={rowidx} data-row={rowidx}>
131                     {row.map((cell, columnidx) => {
132                       const edit = this.state.edit;
133                       if (
134                         edit &&
135                         edit.row == rowidx &&
136                         edit.column == columnidx
137                       ) {
138                         cell = (
139                           <form onSubmit={this.save}>
140                             <input type="text" defaultValue={cell} />
141                           </form>
142                         );
143                       }
144                       return <td key={columnidx}>{cell}</td>;
145                     })}
146                   </tr>
147                 ))}
148               </tbody>
149             </table>
150           </div>
```

```
151          );
152        }
153      }
154      Excell.propTypes = {
155        headers: PropTypes.arrayOf(PropTypes.string),
156        initialData: PropTypes.arrayOf(PropTypes.arrayOf(PropTypes.
    string)),
157      };
158      ReactDOM.render(
159        <Excell headers={headers} initialData={data} />,
160        document.getElementById("app"),
161      );
162    </script>
163  </body>
164 </html>
```

5.8. Conclusiones y Virtual DOM Diffs

En este punto el feature edit esta completo. No tomo mucho codigo

- Mantenimos el tracking de que celda editar via this.state.edit
- Renderisamos el input field cuando mostraba la tabla si el row y la columna coincidian
- Actualizamos el array data con el nuevo valor desde el input field

Tan pronto como tu setState() con el nuevo dato React llama el componente metodo render() y la UI magicamnete se actualiza. Esto podria verse como no es eficiente hacer render de toda la tabla para solo una celda. Es un hecho React solo actualizara una celda Si abres el dev tool puedes ver que partes del DOM tree son actualizados como tu interaccion con tu aplicacion. puedes ver el dev tools highlighting el DOM cambia despues cambiar
Detras de escenas React llama tu metodo render() y crea un lighweight tree representation del DOM result. Esto es conocido como un DOM tree virtual. Cuando el metodo render() es llamado de nuevo (despues una llamada a setSetat())React toma el arbol virtual antes y despues y computa una comparacion Basado en esta comparacion React descrubre las operaciones minimas requeridas DOM(appendChild(), textContent, etc) para llevar los cambios dentro el browser DOM
Hay solo un cambio requerido para la celda y no es necesario renderisar toda la tabla. Calculando el minimo set de cambios y agrupando y computando el DOM, React "toca.[el] DOM ligeramente, como problema conocido que las operaciones DOM son lentas(comparadas a operaciones puras Javacript) llamadas de funciones,etc.
Y amenudo hay cuellos de botellas en el rendimiendo de renderisacion de una web application
React tiene su espalda cuando viene a llevar a cabo y actualizar la UI

- Tocando el DOM ligeramente
- Usando delegacion eventos para interaccion de usuarios

5.9. Search

A continuacion agregamos un search feature a la tabla

1. Agregar un boton para prender el nuevo feature y luego apagarlo
2. Si el search esta prendido Agrega un row de input donde cada busqueda en la columna correspondiente
3. Como un user type en un input box, filtrar el array de state.data para mostrar solo el contenido que coincida

5.10. State y UI

Lo primero a hacer es actualizar el contructor por

- Agregando una propiedad search para el objeto this.state para mantener el trackeo de donde se esta haciendo la busqueda
- Agreando dos nuevos metodos: this.togglesearch() para activar la cajas y apagarlo y this.search() hara la busqueda
- Poniendo una nueva clase this.preSearchData
- Actualizando el dato entrada incicial con un consecutive ID para ayudar a identificar las filas cuando editamos contenidos de datos filtrados

La propiedad search permitira saber si el usuario le dio click a mostrar u ocultar la barra de busqueda mediante el metodo this.togglesearch()
El metodo search hara la busqueda escuchando el texto escrito
Cuando hacemos click en el boton search la funcion toggleSearch() es invocada esta funcion se prende y apaga. Esto es hecho por las tareas

- Seteando el this.state.search en true o false
- Cuando es true, recuerda el dato actual
- Cuando es false revierte al dato

5.11. Filtrando el contenido

El feature search es simple llama al Array.prototype.filter() en el array y retorna un array filtrado con el elmento que conincide el search string. La UI todavia usa this.state.data para hacer el renderizado pero this.state.data es un version reducida de ella misma
Necesitas una referencia al dato antes de la busqueda, entonces no perdereas el dato para siempre. esto permite al usuario volver a la tabla completa o cambiar la busqueda para obtener diferentes coincidencias. Llamemos a esta referencia this.preSearchData. Ahora que hay datos guardados en dos lugares el metodo save() necesitara una actualizacion para ambos campos
Cuando el usuario hace click en el boton search la funcion toggleSearch() es invocada esta funcion prende el search y lo apaga.

- Seteando el this.state.search en true o false
- Cuando es true, recuerda el dato actual
- Cuando es false revierte al dato

```
toggleSearch(){
  if(this.state.search){
    this.setState({
      data: this.preSearchData,
      search: false,
    });
    this.preSearchData = null;
  } else {
    this.preSearchData = this.state.data;
    this.setState({
      search:true
    })
  }
  }
```

La ultima cosa para hacer es implementar el search, cual llama cada ves que algo es escrito en el search row onChange.

```
search(e){
  const needle = e.target.calue.toLowerCase();
  if (!needle){
    this.setState({data: this.preSearchData});
    return;
  }
  const idx = e.target.dataset.idx;
  const searchdata = this. preSearchData.filter((row) => {
    return row[idx].toString().toLowerCase().indexOf(needle) > -1;
  });
  this.setState({data: searchdata});
  }
```

obtienes la busqueda desde los cambios el evento

5.12. Actualizar el metodo save()

Se debe actulizar el metodo save para que soporte el this.preSearchData

```
save(e) {
  e.preventDefault();
  const input = e.target.firstChild;
  const data = clone(this.state.data).map((row) => {
    if (row[row.length - 1] === this.state.edit.row) {
      row[this.state.edit.column] = input.value;
    }
    return row;
  });
  this.logSetState({
    edit: null,
    data,
  });
  if (this.presearchData){
```

```
    this.preSearchData[this.state.edit.row][this.state.edit.column] =
   input.value;
  }
  }
```

5.13. Como podrias mejorar el Search?

Esto fue un simple trabajo de de ilustracion, puedes mejorar el feature para realizar un search de otro search?

5.14. Instant Replay

Como sabes tus componentes se preocupan del estado y React renderisa y renderisa donde sea apropiado. Esto signidifca que dado el mismo dato(state y properties) lucira exactamente igual
Imagina alguien encontrando un bug mientras usa la app, ellos pueden click un boton y reportar el bug sin necesidad de explicar que paso. este bug report puede solo enviarte atras una copia de this.state y this.props y tu seras capas de recrear el estado aplicacion exacta y ver el resultado visual.
un ündo"no seria otro feature, desde React renders your app la misma manera dado el mismo props y estado. es un hecho el ündoïmplementacion es algo trivial: solo necsitas ir atras al estado previo
Tomemos una idea un poco exajerada por diversion. Gravemos cada cambio de estado en el Excel component y luego hagamos un replay. Es facinante ver todas las acciones reproduciendoce. La pregunta de cuando el cambio ocurre no es importante solo reproduscamos el estado de la app
Para implmentar este feature, necesitamos agregar un logSetState() cual primero logea el nuevo estado a un this.log array y despues llama setState() Donde sea en el codigo tu llamas setState() deberia ahora estar cambiada a call logSetState() primero, busca y remplasa todos las llamadas a setState() cual llama a la nueva funcion
Todo lo que era
this.setState()
Ahora sera
this.logSetState();
Ahora movamonos al constructor. Necesitas bindear las dos nuevas funciones log SetState() y replay(), declara esto.log array, y asigna el estado inicial

```
constructor(props) {
  //....
  //log the initial state
  this.log = [clone(this.state)]
  this.replay = this.replay.bind(this)
  this.logSetState = this.logSetState.bind(this);
  }
```

El logSetState necesita hacer dos cosas: log el nuevo estado y despues pasarlo a setState() aqui un ejemplo de implementacion donde haces una cpia del estado

```
        logSetState(newState) {
```

```
          this.log.push(clone(newState));
          this.setState(newState);
        }
```

Ahora que todos los estados estan logueados reproduscamos de vuelta. Para lanzar el playback agregemos un evento simple listener que captura el keyboard action e invoca la funcion replay(). El lugar para event listener como esto es el metodo componenteDidMount() lifecycle:

```
        componentDidMount() {
          document.addEventListener("keydown", (e) => {
            if (e.altKey && e.shiftKey && e.keyCode === 82) {
              // Alt + SHIFT + R(eplay)
              this.replay();
            }
          });
        }
```

Finalmente considrando el metodo replay() es usado setInterval() y una vez por segundo esto lee el siguiente objeto desde el log y pasa a setState():

```
        replay() {
          if (this.log.length === 1) {
            console.warn("No state changes to replay yet");
            return;
          }
          let idx = -1;
          const interval = setInterval(() => {
            if (++idx === this.log.length - 1) {
              clearInterval(interval);
            }
            this.setState(this.log[idx]);
          }, 1000);
        }
```

Y con esto el nuevo feature esta completo puedes ver el ejemplo en 03.11.table.replay.html

5.15. Limpiando manejador de Eventos

El replay necesita un poco de limpieza. Cuando este componente es la unica cosa pasando en la pgina, limpar no es necseario, en una aplicacion real lo componentes son agregados y removidos del DOM frecuentenemente. Cuando removemos desde el DOM una buen ciudadano puede limpar despues de el mismo. En el ejemplo abajo hay dos items que necesitan ser limpiados: el keydown event listener y el replay interval callback Si no limpias el keydown eventlistener fucntion, afectara a la memoria despues del componente se haya ido, Y por que esta usando this, toda la instancia Excell necesita ser retenida en meoria. Esto es memory leak, Muchas de estos y el usuario podria irse y la aplicacion podria crashear el browser tab. Como el intervalo bueno el callback continuara ejecutandoce despues del ocmponete se haya ido y cause otro memory leak. El callback tambien llamara a setState de un componente no existente
Puedes reproducir el error creando un hello world desde la consola

```
ReactDOM.render(
  React.createElement('h1', null, 'Hello world!'),
  document.getElementById('app'),
);
```

Tambien puedes logear un timestamp a la consola en el intervalo callback para ver que esto se mantiene ejecutando

5.16. Cleaning Solution

Tomando cuidado de este filtrado de memoria Necesitas mantener referencia al manejo e intervalo timeouts que quieres limpiar luego limpiarlo en componentWillUnmount() Para el manejador evento teniendo un class method opuesto a un inline function

```
        constructor(props) {
          // ...
          this.replayID = null;
          // ...
          this.keydownHandler = this.keydownHandler.bind(this);
        }
        // .......
        keydownHandler(e) {
          if (e.altKey && e.shiftKey && e.keyCode === 82) {
            // Alt + SHIFT + R(eplay)
            this.replay();
          }
        }
        componentDidMount() {
          document.addEventListener("keydown", this.keydownHandler);
          cleanInterval(this.replayID);
        }
        replay() {
          if (this.log.length === 1) {
            console.warn("No state changes to replay yet");
            return;
          }
          let idx = -1;
          this.replayID = setInterval(() => {
            if (++idx === this.log.length - 1) {
              clearInterval(this.replayID);
            }
            this.setState(this.log[idx]);
          }, 1000);
        }
```

5.17. Puedes mejorar el Replay?

Como implementar un undo/redo feature? cuando la persona usa el Alt + Z bas un paso atras en el stage log y Alt + Shift + Z vas al inicio

5.18. Una Implementacion Alternativa

Hay otra manera de implmentar replay/undo funcionality sin cambiar todo el setState()? Talvez usando una apropiado lifecycle method

5.19. Download los datos de la tabla

Despues de todos el sorting, edigint y searching el usuario esta finalmente feliz con el estado del dato en la tabla. Seria Genial si ellos podrian descargar los datos, el resultado Afortunadamente, no hay nada facil en React Todo lo necesitas hacer es grabar el actual this.state.data y darlo de vuelta por ejemplo en formato JSON o (CSV)
La primera cosa a hacer es agregar nuevas opciones al toolbar(donde esta el Search button) usamos alguna magia HTML para forsar <a>links para lanzar la descarga del archivo

```
        constructor(props){
          ......
          this.downloadJSON = this.download.bind(this, "json");
          this.downloadCSV = this.download.bind(this, "csv");
        }
        download(format, ev) {
          const data = clone(this.state.data).map((row) => {
            row.pop(); // drop the last column, the recordId
            return row;
          });
          const contents =
            format === "json"
              ? JSON.stringify(data, null, " ")
              : data.reduce((result, row) => {
                  return (
                    result +
                    row.reduce((rowcontent, cellcontent, idx) => {
                      const cell = cellcontent.replace(/"/g, '""');
                      const delimiter = idx < row.length - 1 ? "," : ""
    ;
                      return `\${rowcontent}"\${cellcontent}"\${
    delimiter}`;
                    }, "") +
                    "\n"
                  );
                }, "");
          const URL = window.URL || window.webkitURL;
          const blob = new Blob([contents], { type: "text/" + format })
    ;
          ev.target.href = URL.createObjectURL(blob);
          ev.target.download = "data." + format;
        }
        .....
              <div className="toolbar">
                <button onClick={this.toggleSearch}>
                  {this.state.search ? "Hidde search" : "Show search"}
                </button>
                <a href="data.json" onClick={this.downloadJSON}>
                  Export JSON
                </a>
                <a href="data.csv" onClick={this.downloadCSV}>
                  Export CSV
                </a>
              </div>
\end{lstlistign}
\section{Fetching Data}
El componente tiene acceso a los datos en el mimo archivo pero que si
    el dato esta en otro lugar, en un servidor, y necesitas ser obtenido
```

```
   ? Hay varias soluciones para esto y veras mas opciones mas adleante,
    provemos una de las simples obteniendo datos en componentDidMount()
   \\
Digamos que el componente Excell es creado con un initialData property
   \\
El componente puede renderisar un estado intermedio para dejar saber al
    usuario que el dato esta en camino
\begin{lstlisting}[language=html]
       componentDidMount() {
          document.addEventListener("keydown", this.keydownHandler);
          fetch("https://www.phpied.com/files/reactbook/table-data.json
   ")
            .then((response) => response.json())
            .then((initialData) => {
              const data = clone(initialData).map((row, idx) => {
                row.push(idx);
                return row;
              });
              this.setState({ data });
            });
          clearInterval(this.replayID);
        }
       .....
              {this.state.data.length === 0 ? (
                  <tbody>
                    <tr>
                      <td colSpan={this.props.headers.length}>
                        Loading data...
                    </td>
                  </tr>
                </tbody>
              ) : (
                <tbody onDoubleClick={this.showEditor}>
        .....
     ReactDOM.render(
        <Excell headers={headers} initialData={[]} />,
        document.getElementById("app"),
     );
```

Capítulo 6

Functional Excell

Recuerdas function compomponent, En algun punto del capitulo2 tan pronto como el state viene dentro la imagen, los function components dejan caer de la discussion. Es hora de volver a ellos

6.1. Un pequenio recordatorio: Function versus Class Components

En este simple formulario una class component solo necesita un metodo render(). Este es donde construyes la UI, opcionalmente usando this.prop y this.state

```
class Widget extends React.Componet{
    render(){
        let ui;
        return <div>{ui}</div>;
      }
  }
```

En un function component todo el component es la funcion y la UI lo que la funcion retorna. El props es pasado a la funcion cuando el componente es construido

```
function Widget(props){
    let ui
    return <div>{ui}</div>;
  }
```

El menor uso de componentes funciones termina con React v16.8 puedes usarlos solo para componentes que no mantienen un stado (stateless components). Pero con la addicion de hooks en v16.8 ahora es posible usar funciones donde sea a travez de rest de este capitulo veras como el Excell component puede ser implementado como un componente funcion

6.2. Renderisando el Data

El primer paso es renderizar el dato pasado al componente. Como el componente es usado no cambia. En otras plabras un desarrollador usando tu componente no necesita saber si es un componente clase o una funcion. El initialData y headers props parecen iguales. Incluso el propTypes definition son el mismo

```
function Ecel(props){
  }
  Excel.propTypes = {
      headers: PropTypes.arrayOf(PropTypes.string),
      initialData: PropTypes.arrayOf(PropTypes.arrayOf(PropTypes.string
    )),
     }
     const headers = ['Book', 'Author', 'Language', 'Published', 'Sales
    '];
     const data = [
       [
         'A Tale of Two Cities', 'Charles Dickens', // ...
       ],
       //
       ReactDOM.render(
         <Excel headers={headers} initialData={data} />,
         document.getElementById('app'),
       );
```

Implementando este body de la funcion component es copy-paste el body del render() method de la clase component

6.3. The State Hook

Para mantener el estado en los componentes funciones, necesitas hooks. Que es un Hook? Es una funcion con un prefijo * que te deja usar varias React features, como herramientas para manejar estados y componentes lifecycles. Puedes tambien crear tu propio hook. Al final del capiutlo aprendereas como usar muchos build-in hooks como bien escribir tus propios hooks.
Empecemos con el estado hooks. Es una funcion llamada useState() esta disponible como propiedad de React object(React.useState()). Esto toma un valor, el valor inicial de una variable estado(una piesa de dato que quieres manejar), y retorna un array de dos elementos (una tupla. El primero es la variable estado)(una piesa de dato que quieres manejar) y retorna un array de dos elementos(una tupla)El primer elemento es la variable estado y el segundo es una funccion para cambiar esta variable. Veamos un ejemeplo
En la clase componenet, en el contructur definimos el valor inicial

```
hit.state = {
  data:initialData;
  }
```

Despues de eso, cuando quieres cambiar el estado del dato, puedes

```
this.setState({
  data: newData,
  })
```

En una function component, ambos definien el estado inicial y obetiene una funcion updater

```
const [data, setData] = React.useState(initialData);
```